

DEPARTMENT OF TRAINING AND PLACEMENT

Day-4 Exception Handling Practice Questions

Question – 1: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Declare two integer variables:
 - a with value **10**
 - b with value **0**
2. Attempt to divide a by b.
3. Since division by zero is not allowed in Java:
 - Place the division operation inside a try block.
 - Catch the exception using catch (ArithmeticException e).
4. Inside the catch block:
 - Display a user-friendly message "**Cannot divide by zero**".
5. After handling the exception:
 - Print the message "**Program continues...**" to show that the program does not terminate abnormally.

Question – 2: Write a Java program to demonstrate handling exception handling.

Problem Statement:

1. Declare a String variable:
 - s with value null
2. Attempt to find the length of the string using s.length().
3. Since calling a method on a null object causes an exception in Java:
 - Place the statement s.length() inside a try block.
 - Catch the exception using catch (NullPointerException e).

4. Inside the catch block:
 - Display a user-friendly message **"String is null"**.
5. The program should terminate normally after handling the exception without crashing.

Question – 3: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Declare an integer array:
 - arr initialized with the values {10, 20, 30, 40, 50}.
2. Attempt to access an array element using an invalid index.
3. Since accessing an index outside the array size causes an exception in Java:
 - Place the array access statement inside a try block.
 - Catch the exception using catch (ArrayIndexOutOfBoundsException e).
4. Inside the catch block:
 - Display a user-friendly message **"Array index out of range"**.
5. The program should handle the exception gracefully and terminate normally.

Question – 4: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Declare a String value:
 - Use the string "123a".
2. Attempt to convert the string into an integer using Integer.parseInt().
3. Since the string contains non-numeric characters:
 - Place the conversion statement inside a try block.
 - Catch the exception using catch (NumberFormatException e).
4. Inside the catch block:
 - Display a user-friendly message **"Invalid number format"**.
5. The program should handle the exception properly and terminate normally.

Question – 5: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Declare two integer variables and perform a division operation.
2. Place the division statement inside a try block.
3. Handle possible arithmetic errors using:
 - catch (ArithmeticException e).
4. Inside the catch block:
 - Display the message "**Error occurred**".
5. Use a finally block to:
 - Print the message "**Program Ended**".
6. Ensure that the finally block executes whether an exception occurs or not.

Question – 6: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Declare an integer array:
 - Initialize it with the values {1, 2, 3}.
2. Attempt the following operations inside a try block:
 - Access an array element using an invalid index.
 - Perform division by zero.
3. Handle the exceptions using **multiple catch blocks**:
 - Catch ArrayIndexOutOfBoundsException and display "**Array error**".
 - Catch ArithmeticException and display "**Arithmetic error**".
4. Observe that only the first occurring exception is handled.

Question – 7: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Declare an integer array:
 - Initialize it with the values {1, 2}.
2. Attempt to access an array element using an invalid index.
3. Place the risky code inside a try block.
4. Handle multiple exceptions using a **single multi-catch block**:
 - Catch `ArithmeticException` and `ArrayIndexOutOfBoundsException` together.
5. Inside the catch block:
 - Display the message **"Exception handled"**.

Question – 8: Write a Java program to demonstrate nested try–catch blocks

Problem Statement:

1. Write a Java program using **nested try–catch blocks**.
2. In the **inner try block**:
 - Perform a division operation that causes an `ArithmeticException`.
3. Handle the arithmetic error using:
 - `catch (ArithmeticException e)` inside the inner try–catch block.
 - Display the message **"Inner catch: Arithmetic error"**.
4. After the inner try–catch block:
 - Declare an integer array with values {1, 2}.
 - Attempt to access an element using an invalid index.
5. Handle the array error using an **outer catch block**:
 - Catch `ArrayIndexOutOfBoundsException`.
 - Display the message **"Outer catch: Array error"**.
6. Observe how exceptions are handled at different nesting levels.

Question – 9: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Write a Java program that uses:
 - Nested try–catch
 - An outer finally block
2. In the **inner try block**:
 - Perform a valid division operation.
 - Print the result.
3. Handle any arithmetic error using:
 - catch (ArithmeticException e) inside the inner block.
4. Do not use a catch block for the outer try.
5. Use a **finally block** associated with the outer try to:
 - Display the message "**Outer finally executed**".
6. Ensure that the finally block executes regardless of whether an exception occurs or not.

Question – 10: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Write a Java program that performs the following operations:
 - Convert a string into an integer.
 - Divide a number by the converted integer.
2. Declare a String variable:
 - Assign it the value "abc".
3. Attempt to convert the string into an integer using:
 - Integer.parseInt() inside a try block.
4. After conversion, attempt to divide **50** by the converted integer.
5. Handle the possible exceptions using **multiple catch blocks**:

- Catch `NumberFormatException` and display the message "**Invalid number input**".
 - Catch `ArithmeticException` and display the message "**Cannot divide by zero**".
6. Observe that if the string conversion fails, the division operation is not executed.
 7. Ensure that the program terminates normally after handling the exception.

Question – 11: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Write a Java program to demonstrate throwing a built-in exception using the `throw` keyword.
2. Declare an integer variable:
 - Assign it the value 0.
3. Check the value using an if condition.
4. If the value is 0:
 - Manually throw an `ArithmeticException` using the `throw` keyword.
5. Handle the exception using a try-catch block.
6. Display a meaningful error message when the exception is caught.

Question – 12: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Write a Java program to manually throw a `NullPointerException`.
2. Declare a String variable and assign it the value null.
3. Check whether the string reference is null.
4. If the string is null:
 - Use the `throw` keyword to throw a `NullPointerException`.
5. Catch the exception and display a user-friendly message.

Question – 13: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Write a Java program that manually throws an `ArrayIndexOutOfBoundsException`.
2. Declare an integer array with a fixed size.
3. Use an index value greater than or equal to the array size.
4. Before accessing the array:
 - Check the index value.
5. If the index is invalid:
 - Throw an `ArrayIndexOutOfBoundsException` using the `throw` keyword.
6. Handle the exception and print an appropriate message.

Question – 14: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Write a Java program to validate an integer value using **multiple throw statements**.
2. Declare an integer variable:
 - Assign it any value.
3. Check the value using conditional statements:
 - If the value is negative, throw an `ArithmeticException`.
 - If the value is zero, throw an `IllegalArgumentException`.
4. Use the `throw` keyword to throw the appropriate exception based on the condition.
5. Handle the exceptions using multiple catch blocks.
6. Display meaningful error messages for each exception.

Question – 15: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Write a Java program to validate a string input using **multiple throw statements**.
2. Declare a `String` variable.

3. Validate the string using the following conditions:
 - If the string is null, throw a `NullPointerException`.
 - If the string is empty (""), throw an `IllegalArgumentException`.
4. Use the `throw` keyword to throw the appropriate exception.
5. Handle the exceptions using separate catch blocks.
6. Display appropriate validation messages.

Question – 16: Write a Java program to demonstrate exception handling.

Problem Statement:

1. Write a Java program to demonstrate the use of the `throws` keyword.
2. Create a method that performs a file-related operation.
3. In the method definition:
 - Declare that the method **throws `IOException`**.
4. Call the method from the main method.
5. Handle the exception in the main method using a try-catch block.
6. Display the exception message when the exception occurs.